Unit-03

2. Data Types (Collection Data Types)

(a) String

string is a collection of characters enclosed in either single quotes (`'`) or double quotes (`"`) or triple quotes (`'''` or `"""`). Strings are immutable, which means once they are created, they cannot be changed.

1. Creating Strings

You can create a string using single or double quotes.

```
string1 = 'Hello'
print(string1)  # Output: Hello
string2 = "World"
print(string2)  # Output: World
```

2. Multiline Strings

You can use triple quotes (`'''` or `"""`) for multiline strings.

```
string3 = """This is a
multiline string."""
print(string3)
# Output:
 This is a
 multiline string.
```

3. String Immutability

Strings charactor cannot be changed after creation. Instead, you can create a new string.

```
string = "Hello"
# string[0] = "h"  # Error: Strings are immutable
new_string = "h" + string[1:]
print(new_string)  # Output: hello
```

4. Accessing Characters

You can access individual characters in a string using their index.

```
string = "Python"
print(string[0])  # Output: P (1st character)
print(string[-1]) # Output: n (last character)
```

```
print(string[-3]) # Output: h (h character)
```

---

5. String Slicing

Slicing allows you to extract a portion of a string.

```
string = "Programming"
print(string[0:6])   # Output: Progra (from index 0 to 5)
print(string[3:])    # Output: gramming (from index 3 to end)
print(string[:4])    # Output: Prog (from start to index 3)
print(string[-4:])   # Output: ming (last 4 characters)
```

When Slicing Returns an Empty String

A=”Hello”

Condition	Example	Output
`start >= end`	`"Python"[3:3]`	`""`
`start > end` (forward slicing)	`"Python"[5:2]`	`""`
`start` is out of range	`"Hello"[10:15]`	`""`
`end` is too small	`"Hello"[2:-10]`	`""`
Step skips all characters	`"Hello"[1:4:10]`	`"e"`
Wrong step direction	`"Python"[1:4:-1]`	`""`
`step = 0` (Error)	`"Python"[::0]`	`ValueError`

---

6. String Methods

Python provides many built-in methods to work with strings.

```
string = "hello World"
print(string.upper())     # Output: HELLO WORLD
print(string.lower())     # Output: hello world
print(string.title())     # Output: Hello World
```

---

7. String Formatting

a) Using `f-strings` (Python 3.6+)

```
name = "Alice"
age = 25
print(f"My name is {name} and I am {age} years old.")
# Output: My name is Alice and I am 25 years old.
```

b) Using `.format()`

```
print("My name is {} and I am {} years old.".format("Alice", 25))
# Output: My name is Alice and I am 25 years old.
```

---

8. Iterating Through a String

You can iterate through a string using a `for` loop.

```
string = "Python"
for char in string:
    print(char)
# Output:
 P
 y
 t
 h
 o
 n
```

9. Length of a String

The `len()` function gives the number of characters in a string.

```
string = "Python"
print(len(string))  # Output: 6
```

Practice Problems:

Q1: Write a Python program to create three different strings using **single quotes**, **double quotes**, and **triple quotes**. Print them all.

- **Hint:** Use `'single'`, `"double"`, and `'''triple'''` or `"""triple"""`.

Q2: Given the string `word = "Technology"`, print:

The **first** character

The **last** character

The **fourth** character from the end

- **Hint:** Use **indexing** (`word[index]`). Remember, negative indexing can be useful.

Q3: Given the string `text = "Development"`, extract and print:

The **first 5 characters**

The **last 4 characters**
The substring `"velop"`

- **Hint:** Use slicing syntax: `text[start:end]`.

Q4: Write a Python program that takes an input string and prints it **in reverse**.
Example:

```
Input: "Python"
Output: "nohtyP"
```

- **Hint:** Use slicing with a **step of -1** (`[::-1]`).

Q5: Given the string `sentence = "learning PYTHON is Exciting!"`, apply the following methods and print the results:
Convert the string to **uppercase**
Convert the string to **lowercase**
Convert the first letter of **each word** to uppercase

- **Hint:** Use `.upper()`, `.lower()`, and `.title()` string methods.

Q6: Given the string:

```
sentence = "I love programming in Python. Python is amazing!"
```

Replace `"Python"` with `"Java"` and print the new string.

- **Hint:** Use the `.replace(old, new)` method.

Q7: Given the string `"Mississippi"`, count and print how many times the letter `'s'` appears.

- **Hint:** Use the `.count(char)` method.

Q8: Use both **f-strings** and **.format()** to format the following variables into a sentence:

```
name = "Alice"
age = 28
```

Expected Output:

```
My name is Alice and I am 28 years old.
```

- **Hint:** Use:

f-string: `f"My name is {name} and I am {age} years old."`
format(): `"My name is {} and I am {} years old.".format(name, age)`.
Q9: Write a Python program that extracts **only digits** from the string:

```
data = "Python123Is456Fun789"
```

Expected Output:

```
123456789
```

- **Hint:** Use a loop and `.isdigit()` to extract numbers from the string.

Alternatively, use **list comprehension** and `"".join()`.
Q10: Find the first occurrence of a substring

```
text = "Python is amazing, and Python is fun!"
```

Find the first occurrence of `"Python"` using the `.find()` method.

(b) List

list is a collection of different elements that means mixed data types (string, int, float etc..) that is ordered, changeable (mutable), and allows duplicate elements. You can create a list using **square brackets []**.

1. Creating a List

Lists are defined using square brackets `[]`.

```
# Empty list
empty= []
print(empty)  # Output: []

# List with elements
numbers = [1, 2, 3, 4, 5]
print(numbers)  # Output: [1, 2, 3, 4, 5]

# Mixed data types
mixed= [1, "Hello", 3.14, True]
print(mixed)  # Output: [1, 'Hello', 3.14, True]
```

---

2. Accessing List Elements

Use **indexing** to access elements.

```
numbers = [10, 20, -30, 'hello', 40, 50]
print(numbers[0])   # Output: 10 (1st element)
print(numbers[-1])  # Output: 50 (last element)
print(numbers[-3])  # Output: hello
```

---

3. Slicing Lists

Use slicing to access a portion of the list.

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])  # Output: [20, 30, 40] (index 1 to 3)
print(numbers[:3])   # Output: [10, 20, 30] (start to index 2)
print(numbers[2:])   # Output: [30, 40, 50] (index 2 to end)
```

---

4. Modifying a List

Lists are mutable, so you can change their elements.

```
numbers = [10, 20, 30, 40, 50]
numbers[2] = 99
print(numbers)  # Output: [10, 20, 99, 40, 50]
```

---

5. Adding Elements to a List

You can add elements using `append()`, `insert()`, or `extend()`.

a) Using `append()`

Adds an element at the end of the list.

```
numbers = [1, 2, 3]
numbers.append(4)
print(numbers)  # Output: [1, 2, 3, 4]
```

b) Using `insert()`

Adds an element at a specific position.

```
numbers = [1, 2, 3]
numbers.insert(1, 99)  # Insert 99 at index 1
print(numbers)  # Output: [1, 99, 2, 3]
```

b) Using `extend()`

Adds multiple elements to the end of the list.

```
numbers = [1, 2, 3]
numbers.extend([4, 5, 6])  # Extends the list with multiple elements
print(numbers)  # Output: [1, 2, 3, 4, 5, 6]
```

---

6. Removing Elements from a List

You can remove elements using `remove()`, `pop()`, or `del`.

a) Using `remove()`

Removes the first occurrence of a value.

```
numbers = [10, 20, 30, 20, 40]
numbers.remove(20)
print(numbers)  # Output: [10, 30, 20, 40]
```

b) Using `pop()`

Removes and returns an element by index.

```
numbers = [10, 20, 30]
popped = numbers.pop(1)
print(popped)   # Output: 20
print(numbers)  # Output: [10, 30]
```

c) Using `del`

Deletes an element or a slice of elements.

```
numbers = [10, 20, 30, 40, 50]
del numbers[2]
print(numbers)  # Output: [10, 20, 40, 50]

del numbers[1:3]
print(numbers)  # Output: [10, 50]
```

---

7. Sorting a List

You can sort a list using `sort()`

```
numbers = [30, 10, 50, 20]
numbers.sort()
print(numbers)  # Output: [10, 20, 30, 50]

numbers.sort(reverse=True)
print(numbers)  # Output: [50, 30, 20, 10]
```

---

8. Other Useful List Methods

a) `index()`

Finds the index of the first occurrence of a value.

```
numbers = [10, 20, 30, 20]
print(numbers.index(20))  # Output: 1
```

b) `count()`

Counts the occurrences of a value.

```
numbers = [10, 20, 20, 30]
print(numbers.count(20))  # Output: 2
```

c) `reverse()`

Reverses the list in place.

```
numbers = [10, 20, 30]
numbers.reverse()
print(numbers)  # Output: [30, 20, 10]
```

d) `clear()`

Removes all elements from the list.

```
numbers = [10, 20, 30]
numbers.clear()
print(numbers)  # Output: []
```

---

9. Iterating Over a List

You can use a `for` loop to iterate over the list.

```
numbers = [10, 20, 30]
for num in numbers:
    print(num)
# Output:
# 10
# 20
# 30
```

---

10. Nested Lists

Lists can contain other lists.

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matrix[0])      # Output: [1, 2, 3]
print(matrix[1][2])   # Output: 6 (row 2, column 3)
```

---

11. Length of a List

Use `len()` to get the number of elements in a list.

```
numbers = [10, 20, 30]
print(len(numbers))  # Output: 3
```

(c) Tuple

tuple is a collection of different ordered elements that is **immutable** (cannot be changed, added, or removed after creation).Tuples are similar to lists but are typically used when the data should not be modified.

1. Creating a Tuple

Tuples are defined using parentheses `()` or simply by separating items with commas.

```
# Empty tuple
empty_tuple = ()
print(empty_tuple)  # Output: ()

# Tuple with one element (must include a comma)
single_tuple = (5,)
print(single_tuple)  # Output: (5,)

# Tuple with multiple elements
numbers = (1, 2, 3, 4)
print(numbers)  # Output: (1, 2, 3, 4)

# Mixed data types
mixed_tuple = (1, "Hello", 3.14, True)
print(mixed_tuple)  # Output: (1, 'Hello', 3.14, True)
```

---

2. Accessing Tuple Elements

Use **indexing** to access elements in a tuple.

```
numbers = (10, 20, 30, 40)
print(numbers[0])   # Output: 10 (1st element)
print(numbers[-1])  # Output: 40 (last element)
```

3. Slicing Tuples

You can use slicing to extract a portion of a tuple.

```
numbers = (10, 20, 30, 40, 50)
print(numbers[1:4])  # Output: (20, 30, 40)
print(numbers[:3])   # Output: (10, 20, 30)
print(numbers[2:])   # Output: (30, 40, 50)
```

---

4. Immutability of Tuples

Tuples are immutable, so you **cannot modify** their elements after creation.

```
numbers = (10, 20, 30)
```

```
# numbers[1] = 99  # Error: TypeError: 'tuple' object does not support item assignment
```

However, if a tuple contains mutable elements (like a list), those elements can be modified.

```
nested_tuple = (1, [2, 3], 4)
nested_tuple[1][0] = 99
print(nested_tuple)  # Output: (1, [99, 3], 4)
```

---

5. Tuple Concatenation and Repetition

```
tuple1 = (1, 2)
tuple2 = (3, 4)

# Concatenation
result = tuple1 + tuple2
print(result)  # Output: (1, 2, 3, 4)

# Repetition
repeated = tuple1 * 3
print(repeated)  # Output: (1, 2, 1, 2, 1, 2)
```

---

6. Tuple Methods

a) `count()`

Returns the number of occurrences of a value.

```
numbers = (10, 20, 20, 30)
print(numbers.count(20))  # Output: 2
```

b) `index()`

Returns the index of the first occurrence of a value.

```
numbers = (10, 20, 30, 20)
print(numbers.index(20))  # Output: 1
```

---

8. Iterating Over a Tuple

You can use a `for` loop to iterate through a tuple.

```
numbers = (10, 20, 30)
for num in numbers:
```

```
    print(num)
# Output:
# 10
# 20
# 30
```

---

9. Length of a Tuple

Use `len()` to get the number of elements in a tuple.

```
numbers = (10, 20, 30)
print(len(numbers))  # Output: 3
```

---

10. Nested Tuples

Tuples can contain other tuples.

```
nested_tuple = (1, (2, 3), (4, 5, 6))
print(nested_tuple[1])    # Output: (2, 3)
print(nested_tuple[2][1]) # Output: 5
```

11. List Inside Tuple & Tuple Inside List in Python

1. List Inside a Tuple (`tuple` containing `list`)

```
my_tuple = (1, 2, [3, 4, 5], "hello")
print(my_tuple)
```

Output:

```
(1, 2, [3, 4, 5], 'hello')
```

Accessing Elements

```
print(my_tuple[2])  # Output: [3, 4, 5]
print(my_tuple[2][1])  # Output: 4
```

2. Tuple Inside a List (`list` containing `tuple`)

```
my_list = [10, (1, 2, 3), "Python"]
print(my_list)
```

Output:

```
[10, (1, 2, 3), 'Python']
```

Accessing Elements

```
print(my_list[1])  # Output: (1, 2, 3)
print(my_list[1][0])  # Output: 1
```

(d) Dictionary

dictionary is a collection of key-value pairs (items). Key is always unique. It is unordered, mutable, and does not allow duplicate keys.

1. Creating a Dictionary

Dictionaries are created using curly braces `{}`.

```
# Empty dictionary
empty_dict = {}
print(empty_dict)  # Output: {}

# Dictionary with key-value pairs
person = {
    "name": "John",
    "age": 30,
    "city": "New York"
}
print(person)  # Output: {'name': 'John', 'age': 30, 'city': 'New York'}
```

---

2. Accessing Values

Access values using their keys.

```
person = {"name": "John", "age": 30, "city": "New York"}
print(person["name"])  # Output: John
print(person["age"])   # Output: 30

# Using get()
print(person.get("city"))  # Output: New York
```

---

3. Adding and Updating Key-Value Pairs

You can add or update key-value pairs.

```
person = {"name": "John", "age": 30}
person["city"] = "New York"  # Add new key-value pair
print(person)  # Output: {'name': 'John', 'age': 30, 'city': 'New York'}
```

```
person["age"] = 35  # Update existing key
print(person)  # Output: {'name': 'John', 'age': 35, 'city': 'New York'}
```

---

4. Removing Key-Value Pairs

You can remove key-value pairs using `pop()`, `del`.

a) Using `pop()`

```
person = {"name": "John", "age": 30, "city": "New York"}
person.pop("age")
print(person) # Output: {'name': 'John', 'city': 'New York'}
```

b) Using `del`

```
person = {"name": "John", "age": 30}
del person["age"]
print(person)  # Output: {'name': 'John'}
```

```
# Defining the dictionary
person = {"name": "John", "age": 30, "city": "New York"}
# Deleting multiple keys in one del statement
del person["name"], person["age"]
# Printing updated dictionary
print(person)  # Output: {'city': 'New York'}
```

---

5. Iterating Over a Dictionary

You can iterate through keys, values, or both.

a) Keys

```
person = {"name": "John", "age": 30}
for key in person:
    print(key)
# Output:
# name
# age
```

b) Values

```
for value in person.values():
    print(value)
# Output:
# John
# 30
```

c) Key-Value Pairs

```
for key, value in person.items():
    print(f"{key}: {value}")
# Output:
# name: John
# age: 30
```

---

7. Dictionary Methods

a) `keys()`

Returns a view of all keys.

```
person = {"name": "John", "age": 30}
print(person.keys())  # Output: dict_keys(['name', 'age'])
```

b) `values()`

Returns a view of all values.

```
print(person.values())  # Output: dict_values(['John', 30])
```

c) `items()`

Returns a view of all key-value pairs.

```
print(person.items())  # Output: dict_items([('name', 'John'), ('age', 30)])
```

d) `update()`

Merges another dictionary into the current one.

```
person = {"name": "John", "age": 30}
person.update({"city": "New York", "age": 35})
print(person)  # Output: {'name': 'John', 'age': 35, 'city': 'New York'}
```

e) `clear()`

Removes all key-value pairs.

```
person = {"name": "John", "age": 30}
person.clear()
print(person)  # Output: {}
```

---

8. Nested Dictionaries

Dictionaries can contain other dictionaries.

```
nested_dict = {
    "person1": {"name": "John", "age": 30},
    "person2": {"name": "Alice", "age": 25}
}
print(nested_dict["person1"]["name"])  # Output: John
```

---

9. Nested Dictionary with List and Tuple

```
# Defining a nested dictionary with a list and a tuple
student_data = {
    "Alice": {
        "age": 25,
        "subjects": ["Math", "Science", "English"],  # List inside dictionary
        "grades": (90, 85, 88)  # Tuple inside dictionary
    },
    "Bob": {
        "age": 24,
        "subjects": ["History", "Geography", "Economics"],  # List
        "grades": (80, 75, 78)  # Tuple
    }
}
# Accessing values
print(student_data["Alice"]["subjects"])  # Output: ['Math', 'Science', 'English']
print(student_data["Bob"]["grades"])      # Output: (80, 75, 78)
# Accessing specific elements from list and tuple
print(student_data["Alice"]["subjects"][1])  # Output: Science
print(student_data["Bob"]["grades"][0])      # Output: 80
```

Complete Program of List, Tuple and Dictionary

```
# Student details program using list, tuple, and dictionary

# Creating a list of students
students = [
    {"name": "Mahak", "age": 20, "grade": "A", "subjects": ("Math", "Science", "English")},
    {"name": "Ankit", "age": 21, "grade": "B", "subjects": ("History", "Science", "Math")},
    {"name": "Aditya", "age": 22, "grade": "A", "subjects": ("English", "Math", "Physics")}
]

# Display student details
```

```python
for student in students:
    print("\nStudent Details:")
    print("Name:", student["name"])
    print("Age:", student["age"])
    print("Grade:", student["grade"])
    print("Subjects:")
    for subject in student["subjects"]:
        print("-", subject)

# Modifying student details
students[0]["age"] = 23  # Updating age for the first student
print("\nUpdated Age for", students[0]["name"], ":", students[0]["age"])
```

(e) Set

set is a collection of unique element, unordered, and immutable elements. Sets are useful when you need to store non-duplicate items and perform operations like union, intersection, or difference efficiently etc.

1. Creating a Set

Sets are created using curly braces `{}` or the `set()` function.

```python
# Empty set (must use set() because {} creates a dictionary)
empty_set = set()
print(empty_set)  # Output: set()

# Set with elements
numbers = {1, 2, 3, 4}
print(numbers)  # Output: {1, 2, 3, 4}

# Duplicates are automatically removed
numbers = {1, 2, 2, 3, 4, 4}
print(numbers)  # Output: {1, 2, 3, 4}

# Using the set() function with a list
numbers = set([1, 2, 2, 3, 4])
print(numbers)  # Output: {1, 2, 3, 4}

# Set from a string
letters = set("hello")
print(letters)  # Output: {'h', 'e', 'l', 'o'}
```

---

2. Adding Elements to a Set

Use the `add()` method to add a single element to a set.

```
numbers = {1, 2, 3}
numbers.add(4)
print(numbers)  # Output: {1, 2, 3, 4}
```

3. Updating a Set

Use the `update()` method to add multiple elements from an iterable (like a list or another set).

```
numbers = {1, 2, 3}
numbers.update([4, 5, 6])
print(numbers)  # Output: {1, 2, 3, 4, 5, 6}

# Adding elements from another set
numbers.update({7, 8})
print(numbers)  # Output: {1, 2, 3, 4, 5, 6, 7, 8}
```

4. Removing Elements from a Set

You can remove elements using `remove()` or `discard()`.

a) `remove()`

Raises a `KeyError` if the element is not in the set.

```
numbers = {1, 2, 3}
numbers.remove(2)
print(numbers)  # Output: {1, 3}

# numbers.remove(5)  # KeyError: 5
```

b) `discard()`

Does not raise an error if the element is not found.

```
numbers = {1, 2, 3}
numbers.discard(2)
print(numbers)  # Output: {1, 3}

numbers.discard(5)  # No error
print(numbers)  # Output: {1, 3}
```

c) `pop()`

```
numbers = {1, 2, 3}
removed_element = numbers.pop()
print(removed_element)  # Output: 1
print(numbers)          # Output: {2, 3}
```

d) `clear()`

Removes all elements from the set.

```
numbers = {1, 2, 3}
numbers.clear()
print(numbers) # Output: set()
```

---

5. Set Operations

a) Union (`| or union()`)

Combines all unique elements from two sets.

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)  # Output: {1, 2, 3, 4, 5}
# Using union()
print(a.union(b))  # Output: {1, 2, 3, 4, 5}
```

b) Intersection (`& or intersection()`)

Finds common elements between two sets.

```
a = {1, 2, 3}
b = {3, 4, 5}
# Using &
print(a & b)  # Output: {3}
# Using intersection()
print(a.intersection(b))  # Output: {3}
```

c) Difference (`- or difference()`)

Finds elements in one set that are not in the other.

```
a = {1, 2, 3}
b = {3, 4, 5}
# Using -
print(a - b)  # Output: {1, 2}
# Using difference()
print(a.difference(b))  # Output: {1, 2}
```

---

6. Iterating Over a Set

You can iterate over a set using a `for` loop.

```
numbers = {1, 2, 3}
for num in numbers:
    print(num)
# Output:
# 1
# 2
# 3
```

7. Length of a Set

Use `len()` to find the number of elements in a set.

```
numbers = {1, 2, 3, 4}
print(len(numbers))  # Output: 4
```

Comprehensions for All Sequence

Python provides comprehensions as a concise and readable way to construct sequences like lists, sets, dictionaries, and even generators. They allow for faster execution compared to traditional loops.

List Comprehension

Syntax

```
new_list = [expression for VN in iterable if condition]
```

1. Create List and element

```
numbers = []
for i in range(5):
    numbers.append(i)
print(numbers)
Output: [0, 1, 2, 3, 4]
List Comprehension
numbers = [i for i in range(5)]
print(numbers)
Output: [0, 1, 2, 3, 4]
```

2. Squaring Numbers

```
squares = []
for i in range(5):
    squares.append(i**2)
print(squares)
Output: [0, 1, 4, 9, 16]
List Comprehension
squares = [i**2 for i in range(5)]
print(squares)
Output: [0, 1, 4, 9, 16]
```

3. Filtering Even Numbers

```
even_numbers = []
for i in range(10):
    if i % 2 == 0:
        even_numbers.append(i)
print(even_numbers)
Output: [0, 2, 4, 6, 8]
List Comprehension
even_numbers = [i for i in range(10) if i % 2 == 0]
print(even_numbers)
Output: [0, 2, 4, 6, 8]
```

4. Using `if-else` in List Comprehension

```
numbers = []
for i in range(10):
    if i % 2 == 0:
        numbers.append("Even")
    else:
        numbers.append("Odd")
print(numbers)
Output: ['Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even',
'Odd']
List Comprehension
numbers = ["Even" if i % 2 == 0 else "Odd" for i in range(10)]
print(numbers)
Output: ['Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even',
'Odd']
```

5. List of First Letters of Words

```
words = ["apple", "banana", "cherry"]
first_letters = []
for word in words:
    first_letters.append(word[0])
print(first_letters)
Output: ['a', 'b', 'c']
List Comprehension
words = ["apple", "banana", "cherry"]
first_letters = [word[0] for word in words]
print(first_letters)
Output: ['a', 'b', 'c']
```

More Practice Problem

Example 1: Even numbers from a list

```
even_numbers = [x for x in range(10) if x % 2 == 0]
print(even_numbers)  # Output: [0, 2, 4, 6, 8]
```

Example 2: Words longer than 3 letters

```
words = ["cat", "elephant", "dog", "lion"]
```

```
long_words = [word for word in words if len(word) > 3]
print(long_words)  # Output: ['elephant', 'lion']
```

---

3. Using `if-else` in List Comprehension

```
new_list = [expression_if_true if condition else expression_if_false for item
in iterable]
```

Example 3: Mark numbers as Even or Odd

```
labels = ["Even" if x % 2 == 0 else "Odd" for x in range(5)]
print(labels)  # Output: ['Even', 'Odd', 'Even', 'Odd', 'Even']
```

---

4. Nested Loops in List Comprehension

```
new_list = [expression for item1 in iterable1 for item2 in iterable2]
```

Example 4: Cartesian Product (Pairs of numbers)

```
pairs = [(x, y) for x in range(3) for y in range(2)]
print(pairs)  # Output: [(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]
```

---

5. List Comprehension with Functions

Example 5: Applying a function to each element

```
def square(n):
    return n * n

squared_numbers = [square(x) for x in range(1, 6)]
print(squared_numbers)  # Output: [1, 4, 9, 16, 25]
```

---

6. Flattening a Nested List

Example 6: Flattening a 2D list

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat_list = [num for row in matrix for num in row]
print(flat_list)  # Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

---

7. Using List Comprehensions with Dictionaries & Sets

Example 7: Creating a dictionary from a list

```
numbers = [1, 2, 3, 4]
squared_dict = {num: num**2 for num in numbers}
print(squared_dict)  # Output: {1: 1, 2: 4, 3: 9, 4: 16}
```

Example 8: Creating a set with unique values

```
unique_squares = {x**2 for x in [1, 2, 3, 2, 1]}
print(unique_squares)  # Output: {1, 4, 9}
```

Example 9: Converting words to uppercase if they start with 'a'

```
words = ["apple", "banana", "avocado", "cherry"]
filtered_words = [word.upper() for word in words if word.startswith("a")]
print(filtered_words)  # Output: ['APPLE', 'AVOCADO']
```

Example 10: Filtering Multiples of 2 and 3

```
multiples = [x for x in range(20) if x % 2 == 0 and x % 3 == 0]
print(multiples)  # Output: [0, 6, 12, 18]
```

---

2. Set Comprehension

Set comprehensions are used to create sets, ensuring unique values.

Syntax

```
new_set = {expression for item in iterable if condition}
```

Examples

Basic Example

```
squares_set = {x**2 for x in range(5)}
print(squares_set)
# Output: {0, 1, 4, 9, 16}
```

Removing Duplicates

```
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = {num for num in numbers}
print(unique_numbers)
# Output: {1, 2, 3, 4, 5}
```

With Condition

```
even_set = {x for x in range(10) if x % 2 == 0}
print(even_set)
# Output: {0, 2, 4, 6, 8}
```

---

3. Dictionary Comprehension

Dictionary comprehensions create dictionaries efficiently.

Syntax

```
new_dict = {key:value for item in iterable if condition}
```

Examples

Basic Example

```
squares_dict = {x: x**2 for x in range(5)}
print(squares_dict)
# Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Swapping Keys and Values

```
original_dict = {'a': 1, 'b': 2, 'c': 3}
swapped_dict = {v: k for k, v in original_dict.items()}
print(swapped_dict)
# Output: {1: 'a', 2: 'b', 3: 'c'}
```

With Condition

```
filtered_dict = {x: x**2 for x in range(10) if x % 2 == 0}
print(filtered_dict)
# Output: {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

Dictionary from Two Lists

```
keys = ["name", "age", "city"]
values = ["Alice", 25, "New York"]
person_dict = {k: v for k, v in zip(keys, values)}
print(person_dict)
# Output: {'name': 'Alice', 'age': 25, 'city': 'New York'}
```

---

4. Generator Comprehension

Generator comprehensions create iterators instead of storing values in memory.

Syntax

```
generator = (expression for item in iterable if condition)
```

Examples

Basic Example

```
squares_gen = (x**2 for x in range(5))
print(next(squares_gen))  # Output: 0
print(next(squares_gen))  # Output: 1
print(list(squares_gen))  # Output: [4, 9, 16]
```

Using with `sum()`

```
sum_of_squares = sum(x**2 for x in range(5))
print(sum_of_squares)
# Output: 30
```

Efficient Large Dataset Handling

```
large_gen = (x for x in range(10**6))  # Does not store in memory
print(next(large_gen))  # Output: 0
```

---

8. Frozen Set

A **frozenset** in Python is an immutable version of a set. Unlike regular sets, frozensets cannot be modified after creation, meaning you cannot add or remove elements.

Why use frozenset

- **To ensure data remains unchanged** (e.g., configuration settings).
- **To use as dictionary keys** since regular sets are unhashable.
- **To store inside other sets** because sets only accept hashable elements.

Create a frozenset

You can create a **frozenset** using the `frozenset()` function.

```
my_set = {1, 2, 3, 4, 5}
frozen = frozenset(my_set)
print(frozen)  # Output: frozenset({1, 2, 3, 4, 5})
```

Key Properties

1. **Immutable** - Once created, elements cannot be added or removed.
2. **Hashable** - Can be used as a dictionary key or inside another set.
3. **Supports Set Operations** - Can perform union, intersection, difference, etc.
